# Retrieval-Augmented Generation (RAG)

Complete Notes — Concepts, Architecture, Techniques, Evaluation & Production

---

## 1. What Problem Does RAG Solve?

Large Language Models (LLMs) encode knowledge in their parameters at training time. This creates three structural limitations: (1) a fixed **knowledge cutoff** — the model knows nothing about events or data after training; (2) no access to **private/proprietary** data (internal company docs, personal files); (3) a bounded **context window**, so you cannot simply paste an entire knowledge base into every prompt. RAG addresses all three by retrieving only the most relevant external evidence at query time and inserting it into the prompt, so the model generates an answer grounded in that evidence rather than purely from memorized parameters.

**Formal view:** RAG turns generation from P(answer | query) into P(answer | query, retrieved_context), where retrieved_context = Retriever(query, knowledge_base). This is the original formulation from Lewis et al. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks."

## 2. End-to-End RAG Architecture

### 2.1 Offline / Indexing Pipeline

- **Data ingestion:** Pull raw documents from sources (PDFs, HTML, databases, APIs, Confluence, S3, etc.).
- **Parsing / cleaning:** Extract clean text, strip boilerplate, handle tables/images/code blocks appropriately.
- **Chunking:** Split documents into retrievable units (see Section 3).
- **Embedding:** Convert each chunk into a dense vector using an embedding model.
- **Indexing:** Store vectors + metadata (source, page, timestamp, permissions) in a vector store, often alongside a keyword index (BM25) for hybrid search.

### 2.2 Online / Query-Time Pipeline

- **Query understanding:** Optionally rewrite, expand, or decompose the user query.
- **Query embedding:** Embed the (possibly rewritten) query with the same embedding model used at indexing time.
- **Retrieval:** Run similarity search (dense), keyword search (sparse), or both (hybrid) to fetch top-N candidates.
- **Re-ranking:** Optionally re-score the top-N with a cross-encoder or LLM-based ranker to select the best top-k.
- **Context assembly / prompt construction:** Format retrieved chunks (with citations/IDs) into the LLM prompt along with instructions and the user query.
- **Generation:** The LLM produces the final answer conditioned on the assembled context.
- **Post-processing:** Attach citations, verify claims, filter unsafe content, log for observability.

## 3. Chunking Strategies

Chunking quality has an outsized effect on RAG performance — more so than model choice in many cases.

- **Fixed-size (token/character) chunking:** Simple sliding window, typically 256–1024 tokens with 10–20% overlap to avoid losing context at boundaries.
- **Recursive/structural chunking:** Split on natural boundaries first (headings → paragraphs → sentences), falling back to fixed-size only when a unit is still too large (e.g., LangChain's RecursiveCharacterTextSplitter).
- **Semantic chunking:** Compute sentence-level embeddings and group consecutive sentences while similarity stays high, breaking when topic shifts — produces more topically coherent chunks.
- **Document-aware / layout-aware chunking:** Preserve tables, code blocks, and slide structure as atomic units; useful for PDFs, PPTX, and technical documentation.
- **Small-to-big / parent-child chunking:** Embed small chunks for precise retrieval, but return the larger parent chunk/section to the LLM for fuller context (used in LlamaIndex's Auto-Merging Retriever).
- **Sentence-window retrieval:** Retrieve a single sentence, then expand to include a window of surrounding sentences before passing to the LLM.

**Practical trade-off:** Small chunks → more precise retrieval, less context per chunk, more chunks needed. Large chunks → more context but noisier relevance signal and higher token cost. There is no universal ideal size — it should be tuned empirically per corpus and task.

## 4. Embedding Models

- **Purpose:** Map text to a fixed-dimensional dense vector such that semantically similar text is close in vector space (measured by cosine similarity or dot product).
- **Common families:** OpenAI text-embedding-3, Cohere Embed, BGE (BAAI General Embedding), E5, Sentence-BERT / Sentence-Transformers, Google Gecko, Nomic Embed.
- **Domain adaptation:** General-purpose embeddings can underperform on specialized domains (legal, medical, code); fine-tuning or domain-specific embedding models can improve retrieval quality significantly.
- **Dimensionality:** Typically 384–3072 dimensions; higher dimensions can capture more nuance but increase storage and search cost. Matryoshka embeddings allow truncating dimensions with graceful quality degradation.
- **Symmetric vs. asymmetric search:** Symmetric = query and documents are similar in nature (e.g., question-to-question search). Asymmetric = query is short and document is long (typical RAG case) — some embedding models have separate query/passage encoders optimized for this.

## 5. Vector Databases & Indexing Algorithms

### 5.1 Popular Vector Stores

- **Managed / cloud:** Pinecone, Weaviate Cloud, Qdrant Cloud, Zilliz (Milvus).
- **Self-hosted / open-source:** FAISS (library, not a server), Chroma, Milvus, Qdrant, Weaviate.
- **Extensions to existing databases:** pgvector (Postgres), Redis (RediSearch), Elasticsearch/OpenSearch (kNN plugin), MongoDB Atlas Vector Search.

### 5.2 Approximate Nearest Neighbor (ANN) Algorithms

- **HNSW (Hierarchical Navigable Small World):** Graph-based index; excellent recall/speed trade-off, the most widely used ANN algorithm in production vector DBs.
- **IVF (Inverted File Index):** Clusters vectors into cells (via k-means) and searches only nearby cells.
- **PQ (Product Quantization):** Compresses vectors to reduce memory footprint, often combined with IVF (IVF-PQ).
- **ScaNN, Annoy:** Alternative ANN libraries with different speed/accuracy/memory trade-offs.

Exact (brute-force) nearest neighbor search is only feasible for small datasets; production systems with millions+ of vectors rely on ANN indexes that trade a small amount of recall for large speed gains.

## 6. Retrieval Techniques

- **Dense retrieval:** Embedding similarity search — captures semantic/paraphrase matches.
- **Sparse retrieval (BM25/TF-IDF):** Classic term-frequency-based keyword matching — strong on exact terms, IDs, acronyms, rare words that dense models may under-represent.
- **Hybrid search:** Combine dense + sparse scores (e.g., via Reciprocal Rank Fusion) for robustness across query types.
- **Cross-encoder re-ranking:** A second model jointly encodes (query, chunk) pairs for a more accurate but slower relevance score, applied only to the top-N candidates from initial retrieval.
- **Metadata filtering:** Pre- or post-filter by structured fields (date range, document type, access permissions, source) combined with vector search.
- **Maximal Marginal Relevance (MMR):** Re-ranks results to balance relevance with diversity, reducing redundant near-duplicate chunks in the final context.

# 7. Advanced RAG Patterns

## 7.1 Query-Side Techniques

- **Query rewriting:** LLM reformulates an ambiguous or conversational query into a standalone search query.
- **Query expansion:** Generate multiple paraphrased queries to widen recall, then merge/de-duplicate results.
- **Query decomposition:** Break a complex multi-part question into sub-questions, retrieve for each, then synthesize.
- **HyDE (Hypothetical Document Embeddings):** The LLM first drafts a hypothetical answer; that answer (not the raw query) is embedded and used to search, often improving recall since answers resemble other answers/documents more than the original question does.

## 7.2 Retrieval-Generation Loops

- **Multi-hop retrieval:** Alternates retrieval and reasoning steps for questions requiring chained facts (e.g., "Who directed the film that won Best Picture the year X was born?").
- **Self-RAG:** The model is trained/prompted to emit reflection tokens judging whether retrieval is needed, whether the retrieved passage is relevant, and whether the generated answer is supported.
- **Corrective RAG (CRAG):** Evaluates retrieval quality; if retrieved context is judged weak, it triggers a fallback such as web search or query reformulation before generating.
- **Agentic RAG:** An LLM agent autonomously plans which retriever/tool to call, how many times, and when it has enough evidence to answer — blurs into agent architectures (see the companion Agentic AI notes).

## 7.3 Structured / Graph-Based RAG

- **GraphRAG:** Builds a knowledge graph of entities and relations from the corpus; retrieval traverses the graph (or uses graph-derived community summaries) to answer questions requiring relational or global/thematic reasoning that pure chunk-similarity search handles poorly.
- **SQL/Structured RAG:** For tabular/structured data, the LLM generates a SQL query (text-to-SQL) instead of retrieving unstructured chunks.

# 8. Prompt Construction for RAG

- Clearly separate **instructions**, **retrieved context**, and the **user question** using delimiters or structured tags so the model doesn't confuse instructions with retrieved content.
- Include chunk-level **citations/IDs** in the prompt so the model can attribute claims to specific sources in its answer.
- Explicitly instruct the model to **say "I don't know"** or abstain when retrieved context is insufficient, reducing hallucination.
- Order matters: some studies show models attend more strongly to information at the **start and end** of long contexts (the "lost in the middle" effect) — place the most important chunks accordingly.

# 9. Evaluating RAG Systems

## 9.1 Retrieval Metrics

- **Precision@k / Recall@k:** Fraction of retrieved/relevant chunks among top-k results.
- **MRR (Mean Reciprocal Rank):** Rewards ranking the first relevant result higher.
- **nDCG (Normalized Discounted Cumulative Gain):** Accounts for graded relevance and rank position.

## 9.2 Generation / End-to-End Metrics

- **Faithfulness / groundedness:** Is every claim in the answer supported by the retrieved context?
- **Answer relevance:** Does the answer actually address the user's question?
- **Context relevance / precision:** Are the retrieved chunks actually relevant to the query?
- **Context recall:** Did retrieval surface all the information needed to fully answer the question?

## 9.3 Frameworks & Tools

- **RAGAS:** Open-source framework computing faithfulness, answer relevance, context precision/recall using LLM-as-judge techniques.
- **TruLens:** Instrumentation + evaluation for LLM apps, including RAG-specific "triad" metrics.
- **LLM-as-a-judge:** Using a strong LLM to score answers against a rubric or reference — scalable but requires careful prompt design and calibration against human judgments.
- **Human evaluation:** Still the gold standard for nuanced quality, tone, and domain correctness.

## 10. Production & Systems Considerations

- **Latency:** Multi-stage pipelines (query rewrite → retrieve → re-rank → generate) add latency; cache embeddings and frequent queries where possible.
- **Cost:** Embedding + re-ranking + generation calls all incur cost; larger context = more generation tokens billed.
- **Index freshness:** Define a re-indexing strategy (batch nightly vs. streaming/incremental updates) as source documents change.
- **Access control:** Retrieval must respect document-level permissions (e.g., a user should never receive chunks from documents they aren't authorized to see) — enforce via metadata filtering at query time.
- **Observability:** Log queries, retrieved chunk IDs, and generated answers to debug relevance issues and monitor for drift.
- **Guardrails:** Filter retrieved content and generated output for PII, toxicity, and prompt injection originating from untrusted documents (a real risk when documents in the knowledge base are user-supplied).

> **Prompt injection via retrieved documents:** If a knowledge base includes untrusted or externally sourced content, an attacker could embed hidden instructions in a document intended to hijack the LLM's behavior when retrieved. Treat retrieved content as data, not as trusted instructions.

## 11. RAG vs. Fine-Tuning vs. Long-Context / Prompt Engineering

| Approach | Best For | Trade-off |
|---|---|---|
| Prompt engineering | Quick behavior changes, no new knowledge | Bounded by context window & existing model knowledge |
| RAG | External/private/up-to-date knowledge | Infra complexity; retrieval quality bottleneck |
| Fine-tuning | Style, format, deep domain behavior shifts | Costly, can go stale, doesn't reliably add new facts |
| Long-context stuffing | Small, static, well-scoped corpora | Expensive per call; "lost in the middle" effect |

In practice these are complementary: fine-tuning can shape tone/format while RAG supplies facts; RAG systems still benefit from careful prompt engineering to format retrieved context effectively.

## 12. Glossary

**Chunk:** A segment of a source document stored and retrieved as a unit.

**Embedding:** A dense vector representation of text capturing semantic meaning.

**Vector database:** A datastore optimized for similarity search over high-dimensional embeddings.

**ANN (Approximate Nearest Neighbor):** Algorithms (e.g., HNSW, IVF) that trade small recall loss for large speed gains in similarity search.

**Cosine similarity:** A similarity metric based on the angle between two vectors.

**BM25:** A classic sparse/keyword ranking function based on term frequency and inverse document frequency.

**Re-ranking:** A second-pass, higher-precision scoring step applied to initially retrieved candidates.

**Cross-encoder:** A model that jointly encodes a query-document pair for more accurate relevance scoring than embedding similarity alone.

**HyDE:** Hypothetical Document Embeddings — using a generated hypothetical answer to drive retrieval.

**Groundedness / Faithfulness:** The degree to which a generated answer is actually supported by retrieved context.

**Hallucination:** Plausible-sounding but factually incorrect or unsupported model output.

**Knowledge graph:** A structured representation of entities and their relationships, used in GraphRAG.

**Context window:** The maximum number of tokens an LLM can process in a single call.

---

End of RAG notes.